

## File Handling in Python

File handling in Python allows you to work with files on your system. Python provides built-in functions and methods for reading, writing, and manipulating files. The core operations include opening a file, reading from it, writing to it, and closing it. Python's built-in `open()` function provides a way to interact with files.

### Basic Steps in File Handling

1. **Opening a File**
2. **Reading from a File**
3. **Writing to a File**
4. **Closing a File**

#### 1. Opening a File ( `open()` function)

To work with a file, the first step is to open it. The `open()` function is used to open a file. It takes two arguments:

- **filename:** The name of the file you want to open.
- **mode:** The mode in which to open the file. Modes define how the file should be handled (read, write, etc.).

##### Common Modes:

- `'r'`: Read (default mode) - Opens the file for reading. The file must exist.
- `'w'`: Write - Opens the file for writing. If the file exists, it is overwritten. If it does not exist, it is created.
- `'a'`: Append - Opens the file for appending. Data is written to the end of the file without overwriting existing content.
- `'b'`: Binary - Used for binary files.
- `'x'`: Exclusive creation - Creates a new file. If the file already exists, it raises an error.

Example:

```
file = open('example.txt', 'r') # Open the file for reading
```

#### 2. Reading from a File

Once a file is opened in **read mode**, you can read its contents using the following methods:

**`read()`:** Reads the entire file as a single string.

```
file = open('example.txt', 'r')
content = file.read()
print(content)
file.close()
```

**readline()**: Reads the next line of the file.

```
file = open('example.txt', 'r')
line = file.readline() # Reads the first line
print(line)
file.close()
```

### 3. Writing to a File

To write to a file, the file must be opened in **write ('w')** or **append ('a')** mode.

**write()**: Writes a string to the file. If the file is opened in write mode ('w'), it overwrites the existing content.

```
file = open('example.txt', 'w') # Open the file in write mode
file.write("Hello, World!")
file.close()
```

**writelines()**: Writes a list of strings to the file.

```
lines = ["Line 1\n", "Line 2\n", "Line 3\n"]
file = open('example.txt', 'w')
file.writelines(lines)
file.close()
```

### 4. Closing a File

It's important to **close** the file after performing operations to free up resources. The **close()** method is used to close the file.

```
file = open('example.txt', 'r')
content = file.read()
file.close() # Close the file after reading
```

### Using **with** Statement (Context Manager)

A more efficient way of handling files is by using the **with** statement. This automatically closes the file once the block of code is executed, even if an error occurs. It's a safer way to handle files because you don't have to explicitly call `close()`.

```
with open('example.txt', 'r') as file:
    content = file.read()
    print(content)
# The file is automatically closed when exiting the 'with' block
```

## Example of Complete File Handling (Read, Write, Append)

Let's walk through an example where we:

- Open a file in **write** mode to write some data.
- Open the file again in **read** mode to read the content.
- Open the file in **append** mode to add more data.

# Write to a file

```
with open('example.txt', 'w') as file:
    file.write("This is the first line.\n")
    file.write("This is the second line.\n")
```

# Read from the file

```
with open('example.txt', 'r') as file:
    content = file.read()
    print("File Content After Writing:")
    print(content)
```

# Append to the file

```
with open('example.txt', 'a') as file:
    file.write("This is an appended line.\n")
```

# Read the updated file

```
with open('example.txt', 'r') as file:
    updated_content = file.read()
    print("File Content After Appending:")
    print(updated_content)
```

## Error Handling with File Operations

It's a good practice to handle errors that may occur during file operations. You can use **try-except** blocks to catch exceptions like **FileNotFoundError** or **IOError**.

```

try:
    with open('non_existent_file.txt', 'r') as file:
        content = file.read()
except FileNotFoundError:
    print("The file was not found.")
except IOError:
    print("An error occurred while reading the file.")

```

## File Operations Summary

| Operation           | Mode | Description                                                       |
|---------------------|------|-------------------------------------------------------------------|
| <b>open()</b>       | 'r'  | Opens a file for reading (default mode).                          |
|                     | 'w'  | Opens a file for writing, creates the file if it doesn't exist.   |
|                     | 'a'  | Opens a file for appending, creates the file if it doesn't exist. |
|                     | 'b'  | Opens a file in binary mode.                                      |
|                     | 'x'  | Creates a new file, raises an error if file exists.               |
| <b>read()</b>       | N/A  | Reads the entire file content.                                    |
| <b>readline()</b>   | N/A  | Reads the next line of the file.                                  |
| <b>readlines()</b>  | N/A  | Reads the entire file and returns lines as a list.                |
| <b>write()</b>      | 'w', | Writes a string to the file.                                      |
|                     | 'a'  |                                                                   |
| <b>writelines()</b> | 'w', | Writes a list of strings to the file.                             |
|                     | 'a'  |                                                                   |
| <b>close()</b>      | N/A  | Closes the file to free up resources.                             |
| <b>with</b>         | N/A  | Automatically handles opening and closing the file.               |

## Conclusion

File handling in Python is straightforward but important for reading and writing data to files. Using the `open()` function, various modes, and the `with` statement allows you to interact with files easily. Always handle errors gracefully to avoid unexpected issues and ensure that files are closed after use.

## File Position

Every file in Python maintains a **file pointer** that indicates the current position within the file. When you read or write to a file, the file pointer moves accordingly.

### Methods to Control File Position:

- **tell()**: Returns the current position of the file pointer.
- **seek(offset, whence)**: Moves the file pointer to the specified position.
  - **offset**: The number of bytes to move the pointer. A positive number moves the pointer forward, and a negative number moves it backward.
  - **whence** (optional): Defines where the offset is applied:
    - **0**: From the beginning of the file (default).
    - **1**: From the current position.
    - **2**: From the end of the file.

### Summary of File Handling Operations:

1. **Opening a File**: You unlock the file in a specific mode (read, write, append).
2. **Reading from a File**: You extract data from the file, either fully or line by line.
3. **Writing to a File**: You write new data to the file, either replacing old data or appending new data.
4. **Closing a File**: You close the file to ensure that changes are saved and system resources are freed.
5. **File Position**: The file keeps track of your current location as you read or write. You can move the pointer around to read or write at different places in the file.